FastMACE: Efficient GPU Inference and Molecular Dynamics for Pretrained MACE Potentials

Fabian Zwingel
University of Bayreuth, Germany

Draft of 11 November 2026

Abstract

MACE potentials are accurate but costly to evaluate: a general implementation runs each layer as separate kernels, carries large tensors for every neighbour pair, and returns to the host at every molecular dynamics step. FastMACE runs *unchanged pretrained checkpoints* faster by reorganising the computation while keeping MACE's own PyTorch modules. It uses the frozen parameters to move linear work out of the neighbour loops, replaces the radial networks with piecewise-polynomial tables that must pass a numerical test at setup, streams the second interaction and its derivative without storing per-edge weights, runs the dense products on tensor cores, and executes complete MD steps on the GPU as captured CUDA graphs. A tiled evaluator with a wavefront schedule bounds device memory for large systems. For the OMAT-0 medium model on 5,000 atoms of $SrTiO_3$, recorded Langevin steps take 3.4 ms on an NVIDIA H200 and 6.2 ms on an A100. In same-machine runs on the H200 a step took 5.7 ms against 14.6 ms for Symmetrix-XL 0.1.2, the closest engine. With the inter-layer state in host memory, 180 million atoms have been evaluated on a 20 GB GPU. On the tested structures, force and stress errors stay within 1.3 times the float32 error of the reference implementation, under an acceptance limit of three. We give a paired measurement for each mechanism, a kernel-time profile, the origin of each idea, and the gaps that remain: no downstream validation, one GPU, and no benchmark of a single revision.

Keywords: machine-learned interatomic potentials, MACE, equivariant neural networks, GPU kernels, molecular dynamics, tensor cores

1 Introduction

A molecular dynamics (MD) simulation evaluates the same trained potential at every step, with fixed parameters and slowly changing geometry. A general neural-network implementation does not use that structure. For MACE [1], a force call expands radial weights for every neighbour pair, forms tensor products edge by edge, keeps the results for automatic differentiation, and returns to a host-side integrator and neighbour list before the next step.

FastMACE is a library that runs pretrained MACE checkpoints on NVIDIA GPUs with these costs removed where the mathematics allows it. Its unit of optimisation is the complete energy, force and MD-step calculation, not a single layer. No retraining, distillation or change of time step is involved.

Contributions.

- A rewrite of the first interaction that sums radial features over neighbours before projecting onto the channels, with fused forward and reverse kernels (Section 4).

- Radial tables with an explicit acceptance test against the float32 network's own error, and a packed eight-byte form for the forward pass (Section 5).
- A reverse kernel for the streamed second interaction that takes weights and slopes from the table in registers, inside PyTorch's automatic differentiation (Section 6).
- Tensor-core dense products, including a hybrid kernel that needs two tensor-core products where the standard method needs three (Section 7).
- A GPU-resident MD step: two captured graphs, launched before the host has read the previous flags, with the neighbour rebuild beside the force call (Section 8).
- A tiled evaluator that stores only what the next interaction reads, releases it along a wavefront, and can keep it in host memory (Section 9).
- For each of these, a paired timing in both build orders, a hardware-counter profile of the result, a numeric acceptance rule, and a record of where the idea came from (Sections 10–13).

Closest work. Symmetrix-XL [2], published on 1 October 2026, has the same aim. It converts the checkpoint into a native evaluator with kernels generated for the model, streams edges, replaces the radial networks by splines, and evaluates large systems tile by tile in three sweeps. It runs under LAMMPS and ASE, on CPUs and on many GPUs, and it reproduces published downstream benchmarks. FastMACE adopted three of its ideas (Table 12) and differs in keeping MACE's PyTorch modules, in its own MD engine, in tensor-core arithmetic and in how the tiled path stores its state. We compare with it wherever we have a measurement (Section 11.3) and say where it is ahead (Section 14).

Limits of the evidence. The paper describes the mechanisms for one concrete case, the OMAT-0 medium model in float32. The step times come from separate adoption studies on different dates, not from one benchmark of a single revision. The comparisons with other engines were made with earlier versions of FastMACE. The numerical checks are sampled tests on small cells, not bounds, and no downstream property has been validated.

Several of the mechanisms were first proposed elsewhere: in the literature, in other inference engines, or in research reports produced by large language models at the author's request. Section 13 records the origin of each.

2 The calculation being accelerated

Model. MACE [1] builds on the atomic cluster expansion [3] and equivariant message passing [4, 5]. For positions R, cell H and species z the energy is

$$E(R,H,z) \;=\; \sum_i E^0_{z_i} \;+\; \sum_i \varepsilon_i(R,H,z) \;+\; E_{\mathrm{pair}}(R,H,z), \qquad F_i = -\frac{\partial E}{\partial R_i}, \tag{1}$$

where ε_i is the learned site energy and E_{pair} a short-range repulsion. Stress follows from the strain derivative.

For a directed edge from sender j to receiver i let r_{ij} be its length and $Y_b(\hat{r}_{ij})$ its real spherical harmonics. An interaction forms the message

$$m^{(p)}_{i,o,c} \;=\; \sum_{j\in\mathcal{N}(i)} w_{p,c}(r_{ij}) \sum_{a,b} C^{(p)}_{oab}\, x_{j,a,c}\, Y_b(\hat{r}_{ij}), \tag{2}$$

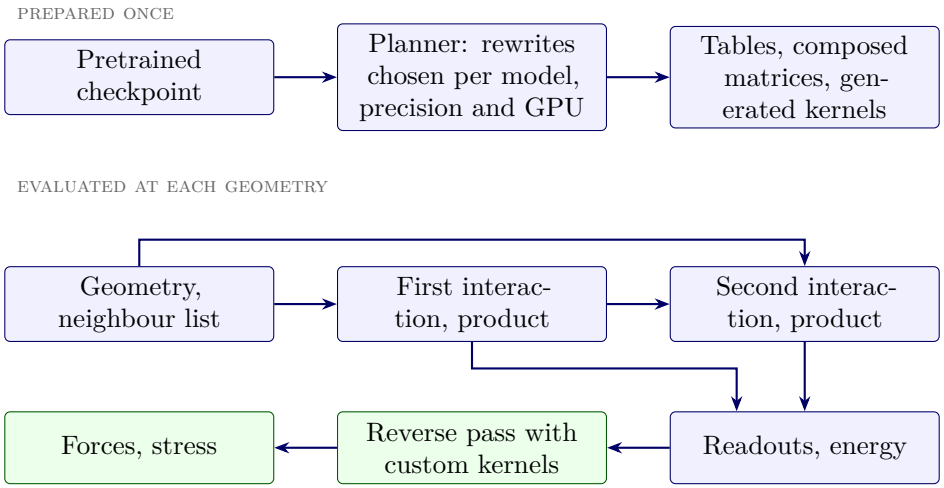


Figure 1: Structure of FastMACE. Preparation is allowed to be expensive because its products are reused. The repeated calculation is captured in CUDA graphs. A tiled evaluator (Section 9) reuses the same rewritten modules with a different storage schedule.

with channel c, coupling path p, input and output components a and o, Clebsch–Gordan coefficients C, and radial weights w produced by a small network of the distance (and of the two species). A symmetric contraction then forms polynomial combinations of the message at each atom, equivariant linear maps mix channels, and readouts give ε_i.

The example model. All details below refer to OMAT-0 medium, a two-interaction MACE foundation model [6] trained on the OMat24 data [7]. It has 128 channels, harmonics up to degree 3 (16 components), a 6 Å cutoff, and symmetric contractions up to third order. The second interaction receives 512 features per atom and has 10 paths, so each edge carries $10 \times 128 = 1{,}280$ radial weights. Its final readout contains a nonlinear layer.

Where a general implementation spends its work. The benchmark used throughout is $SrTiO_3$, $10 \times 10 \times 10$ cells, 5,000 atoms and about 370,000 directed edges (74 per atom in the perfect crystal). Storing the second interaction's weights alone takes $370{,}000 \times 1{,}280 \times 4$ bytes ≈ 1.9 GB per force call, and their gradient the same again. The standard MD loop adds a host-side integrator and a neighbour search at every step, with two transfers between host and device. Symmetrix-XL uses the same model, crystal, precision, cutoff and skin for its throughput and capacity measurements [2], which makes its published numbers a useful reference point.

3 Design

FastMACE keeps MACE's own PyTorch modules and rewrites a private copy of the model. Engines such as Symmetrix-XL instead convert the checkpoint into their own representation [2]; Table 11 in Section 12 sets the two designs side by side. Figure 1 shows the structure.

Rewrites and their classes. Each change is a *rewrite* that the planner turns on for a model, precision and GPU. Every rewrite belongs to one of three classes, which the paper keeps apart:

- **Identity in real arithmetic**: composing linear maps, moving a linear map through a sum. Only rounding changes.

- **Rounding change**: a different summation order or a tensor-core decomposition of a product.
- **Sampled approximation**: radial tables and packed coefficients, accepted by a numerical test at setup.

Some rewrites assume frozen parameters and provide no training gradients.

How defaults are chosen. A rewrite becomes a default for a GPU family only after a paired timing in one process, with alternating blocks and both build orders, shows it faster in both, and after the accuracy rule of Section 10 passes. The noise floor of this comparison was measured at 0.2 % to 1.1 % depending on the machine. A rewrite that only removes work is enabled everywhere after one such test. Defaults therefore differ between GPUs (Table 2).

Software. The fast path is a PyTorch computation with specialised operators. PyTorch's compiler [8] fuses elementwise regions; Triton [9], run-time compiled CUDA, CUTLASS [10] and cuEquivariance [11] implement individual operations; automatic differentiation connects custom reverse kernels to the rest. Appendix A lists the versions.

4 Rewrites that use frozen parameters

4.1 Aggregate before projecting in the first interaction

In the first interaction the sender features depend only on the species. Together with the fact that the last layer of the radial network is linear, this allows the projection onto the 128 channels to move outside the neighbour sum. With $a_k(r)$ the hidden radial features (64 per species pair, cutoff included),

$$S_{i,z,b,k} = \sum_{j \in \mathcal{N}(i),\ z_j = z} a_k(r_{ij}; z_i, z)\ Y_b(\hat{r}_{ij}), \tag{3}$$

$$m_{i,b,c} = \sum_{z,k} W^{(b)}_{z,c,k}\ S_{i,z,b,k}, \tag{4}$$

where W combines the frozen radial matrix, the species features and the path normalisation. The edge loop then accumulates 16×64 numbers per receiver and sender species, and the projection is a dense product per atom. This is the density trick of the atomic cluster expansion [3] applied to MACE's first layer. Later interactions have environment-dependent sender features and cannot use it; there, Symmetrix-XL applies a learned linear map once per receiver after the neighbour sum [2], which is a related but different move.

The implementation evaluates the radial table inside the edge walk and never writes an edge-by-feature array. Its reverse recomputes values and slopes, contracts the local adjoint into a distance derivative at once, and discards it. In the profile of Table 1 the first interaction's forward and reverse kernels take 323 μs of a 3,258 μs force call on the H200. In our profile of Symmetrix-XL 0.1.2 on the same GPU the first interaction took 6.4 ms (Figure 5), which is where most of the difference between the two engines arose on that date.

4.2 Composing adjacent linear maps

With frozen parameters, adjacent linear maps can be multiplied during preparation. FastMACE folds the linear map after the first product into its three consumers (the second interaction's

input projection, its species-dependent skip map and the first readout), composes the first interaction's projection with its linear map and species skip, and composes the map into the final readout with the preceding product, so that the readout tail runs at 16 instead of 128 channels. Nonlinearities stay in place.

4.3 A generated symmetric contraction

The first symmetric contraction is a fixed polynomial for a given checkpoint: 16 angular inputs and four outputs per channel in OMAT-0 medium. FastMACE generates a straight-line program for it from the contraction's descriptor, with repeated products shared, together with its reverse. A factored variant evaluates the same polynomial through shared pair products in a basis of 13 scalar invariants and 16 vector covariants, with the species-dependent weights transformed at setup. The division by the neighbour density that follows the first interaction is fused into the same kernel. On the H200 the four kernels this replaces took 179.5 μs per force call and the generated pair takes 82.4 μs. This is the only kernel FastMACE generates per model; Symmetrix-XL generates its Clebsch–Gordan contractions per checkpoint as a matter of design.

5 Radial functions as tested tables

A frozen radial network is a function of one variable per species pair. FastMACE evaluates it in float64 at setup and stores a piecewise polynomial, which removes the radial embedding, the hidden layers and the cutoff from the force loop. Tabulating an embedding network is established practice for Deep Potential models [12], and Symmetrix-XL projects MACE's radial networks onto cubic Hermite splines [2]. The forms used here follow from what each interaction consumes.

First interaction. It needs the hidden features $a_k(r)$. They are stored as Chebyshev series [13] of degree 5, together with the cutoff and density terms. The pieces start as eight uniform intervals and are halved until values and derivatives at test points agree with the network to 10^{-7} of their scale. For the nine species pairs of $SrTiO_3$ this gives 619 pieces and a 15 MB table.

Second interaction. It reads the 1,280 weights directly from cubic Hermite splines, evaluated in the power basis,

$$w(r) \approx c_0 + t\,(c_1 + t\,(c_2 + t\,c_3)), \qquad t = (r - r_k)/h_k. \tag{5}$$

This removes the hidden-to-weight matrix product from every force call. The knots are uniform within bands and denser at short distances; the recorded tables have 731 to 736 intervals between 0.3 and 6 Å, about three quarters of them below 2.5 Å. Reading the weights from splines in a receiver-major kernel follows Symmetrix-XL, which uses 256 nodes for this model [2]. The banded knots and the acceptance test are ours.

The acceptance test. A table is used only if it passes the following test at setup; we call a table that passed it *certified*. For each species pair, 6,000 uniform and 6,000 random distances are drawn. On these, the float32 table and the float32 network are both compared with the float64 network, for values and for first derivatives, each normalised by the largest value of its output column. The table's largest error must not exceed the network's own, over the whole range and over distances from 1 Å separately. The recorded worst ratios are 0.36 for the Chebyshev table and 0.49 for the spline (its slopes; 0.01 for its values). This is a sampled test,

not a bound for every distance. Symmetrix-XL gates its splines in a comparable way, on node values, interpolation error, derivatives and end-to-end energies, forces and stresses [2].

A packed table for the forward pass. On the forward pass the table is the dominant stream of bytes. An optional packed form stores eight bytes per weight instead of sixteen: the cubic is replaced by a quadratic in a centred coordinate, with three fixed-point coefficients of 22, 20 and 16 bits and a shared six-bit exponent. Its values pass the test at a ratio of 0.16. Its slopes would fail by three orders of magnitude, so the reverse pass keeps the float32 cubic. *The forces are then not the exact gradient of the reported energy.* Both remain within the acceptance rule of Section 10, but the two are no longer consistent to round-off, and we have no energy-drift record for this path yet (Section 14). Symmetrix-XL does not have this problem: its forces are the derivative of the same spline that gives its energy.

6 Streaming the second interaction

Forward. Work is assigned to receiver-and-channel pairs. One thread walks a receiver's edges, evaluates the radial weights, contracts sender features with the harmonics, and accumulates the 40 output components of its channel in registers. The result is written once, already in the block layout of the following linear maps. Neither the edge messages nor the radial weights are materialised. Edges are staged in tiles of 32 in shared memory, and each receiver's edges are sorted by length so that neighbouring receivers read the same table pieces.

The kernels descend from the receiver-streaming convolution of Fuchs [14] in OpenEquivariance [15], which adapts the schedule of Sobek [16]; the adapted files keep the upstream licence. The receiver-major layout with spline weights was taken from Symmetrix-XL after a profile on 6 November showed its forward at 952 μs against 1,623 μs for ours on the H200; with that layout ours takes 629 μs (Table 1).

Reverse without stored weights. For one edge, path and channel let g_o be the adjoint of the receiver's message. With

$$u_a = \sum_{o,b} C_{oab}\, g_o\, Y_b, \qquad v_b = \sum_{o,a} C_{oab}\, g_o\, x_a, \qquad q = \sum_{a} x_a u_a, \tag{6}$$

the three adjoints are

$$\bar{x}_a \mathrel{+}= w\, u_a, \qquad \bar{Y}_b \mathrel{+}= w\, v_b, \qquad \bar{r} \mathrel{+}= w'\, q. \tag{7}$$

Algorithm 1 gives the kernel. It rebuilds w and its slope w' from the coefficients of Eq. (5), so the edge-weight gradient, a tensor of 1.9 GB in the general implementation, and the dense projection through it never exist. The receiver's adjoint row is copied to shared memory once per run of edges.

Not storing per-edge weights in the reverse pass is not new: Symmetrix-XL's reverse pass also recomputes its radial and angular factors [2]. What is specific here is the setting. The kernel runs as one node of PyTorch's automatic differentiation, so the rest of the model keeps its ordinary reverse pass. Its sums for $\bar{x}$ and $\bar{Y}$ are those of the vendored OpenEquivariance kernel; our change replaces that kernel's store of the weight gradient by the product with w' in registers.

What limits these kernels. Table 1 shows that the second interaction is about 53 % of the kernel time of a force call. On the A100 both of its kernels are bound by the fused multiply-add units (84–85 % busy); on the H200 they are bound by instruction issue. This shapes what further changes can gain (Section 14).

Algorithm 1 Reverse pass of the second interaction for one edge $e = (j \to i)$ and channel c. The adjoint g of the receiver's message is read from a row staged in shared memory.

find the table interval k of r_e; $\quad t \leftarrow (r_e - r_k)/h_k$; $\quad \bar{r} \leftarrow 0$
for each path p **do**
  $(c_0, c_1, c_2, c_3) \leftarrow \mathrm{table}[k, p, c]$
  $w \leftarrow c_0 + t\,(c_1 + t\,(c_2 + t\,c_3))$ ▷ value
  $w' \leftarrow (c_1 + t\,(2c_2 + 3t\,c_3))/h_k$ ▷ slope from the same coefficients
  $u_a \leftarrow \sum_{o,b} C^{(p)}_{oab}\, g_{i,o,c}\, Y_b(\hat{r}_e)$
  $v_b \leftarrow \sum_{o,a} C^{(p)}_{oab}\, g_{i,o,c}\, x_{j,a,c}$
  $\bar{x}_{j,a,c}$ += $w\, u_a$ ▷ atomic add into the sender's row
  $\bar{Y}_{e,b}$ += $w\, v_b$
  $\bar{r}$ += $w' \sum_a x_{j,a,c}\, u_a$
end for
add $\bar{r}$ to the distance adjoint of edge e

	H200		A100-SXM4-80GB	
Component	μs	share	μs	share
Second interaction, reverse	1,089	33 %	2,256	35 %
Second interaction, forward	629	19 %	1,255	19 %
Dense products (tensor cores)	609	19 %	1,069	16 %
First interaction, forward and reverse	323	10 %	844	13 %
Symmetric contractions	192	6 %	402	6 %
Other (geometry, elementwise, reductions)	416	13 %	686	11 %
Total	3,258		6,512	

Table 1: Kernel time of one force call (OMAT-0 medium, $SrTiO_3$, 5,000 atoms), from a hardware-counter profile of 9 November 2026. The profile predates the generated contraction on the H200 and the hybrid product on the A100.

7 Dense products on tensor cores

After the edge work, dense per-atom products are the next largest cost. Tensor cores multiply at reduced precision, so a float32 operand is split into a high part and a residual, and three reduced-precision products recover the high–high term and the two cross terms [17]. The residual–residual term is dropped. FastMACE uses the CUTLASS kernel for this [10], with the division by the density and the species selection fused into its output stage.

On the A100 a hybrid variant computes the high–high term in TF32 and packs both cross terms into one half-precision product,

$$AB \;\approx\; A_h B_h \;+\; 2^{-11}\,[\,A_h \;\; A'_l\,]\begin{bmatrix} B'_l \\ B_h \end{bmatrix}, \qquad A'_l = 2^{11}(A - A_h), \tag{8}$$

with float32 accumulation. This takes two tensor-core instructions per step instead of three. Its error is at or below that of the three-product kernel on every tested shape, and it ran at 0.87 of that kernel's time summed over nine representative shapes. The operands must stay below 32,752 in magnitude; the recorded operands of this model are far below, but there is no run-time guard.

GPU	Dense products	First symmetric contraction
H200	CUTLASS, three TF32 products	generated
A100-SXM4-80GB	CUTLASS, hybrid TF32 / FP16	generated, factored
RTX 4000 Ada	Triton, three TF32 products	generated, factored
RTX 5060 Ti	float32	cuEquivariance

Table 2: Routes chosen by measurement for OMAT-0 medium in float32. The differences reflect which unit limits each GPU.

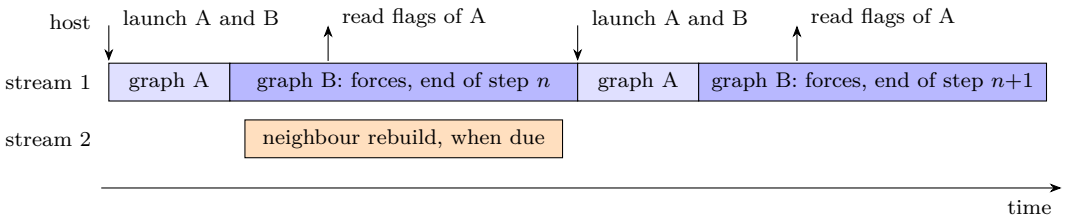


Figure 2: One MD step as two captured graphs (schematic, not to scale). Graph A integrates, checks displacements and compacts the neighbour candidates; graph B evaluates forces and completes the integration. The host launches both before it reads the flags of A, and a rebuild of the neighbour list runs on a second stream beside the force evaluation.

8 Executing complete MD steps

Positions, velocities and forces stay on the GPU. A step replays two captured CUDA graphs [18]: the first integrates, checks displacements and compacts the neighbour candidates to the cutoff; the second evaluates forces and completes the integration (Figure 2). Graphs are captured for padded edge counts, and padding contributes exactly zero. The engine provides Langevin, velocity-Verlet, stochastic-rescaling, Nosé–Hoover-chain and barostat integrators behind classes that follow ASE's interface [19].

Pipelining. The engine launches the force graph before the host has read the flags of the first graph. If the edge count has outgrown the capacity, or the neighbour list has expired, the engine restores the saved mid-step state and repeats the force stage with the same random numbers. The profiled runs had no repeated steps. The measured gaps between kernels were 63 μs in a 3.24 ms force graph on the H200.

Neighbour list. The list holds pairs within the cutoff plus a skin [20] and is rebuilt on the GPU. In fixed-cell runs the rebuild starts early, on a second stream, beside the force evaluation. For more than 10,000 atoms a cell-list screen replaces the pairwise screen of tiles (Section 11.5). Langevin noise is drawn in batches of 256 steps on a background thread.

None of this changes the time step, skips a force evaluation or reuses stale features.

9 Tiled evaluation for large systems

The whole-system path keeps every intermediate of a force call in device memory. For larger systems FastMACE partitions the atoms into receiver tiles; a tile owns the edges that end at its receivers and gathers the sender halo they need. Evaluating a two-interaction model tile by tile in three sweeps, with only the inter-layer state global, is the tiled mode of Symmetrix-XL [2], which we follow. The following parts are our own.

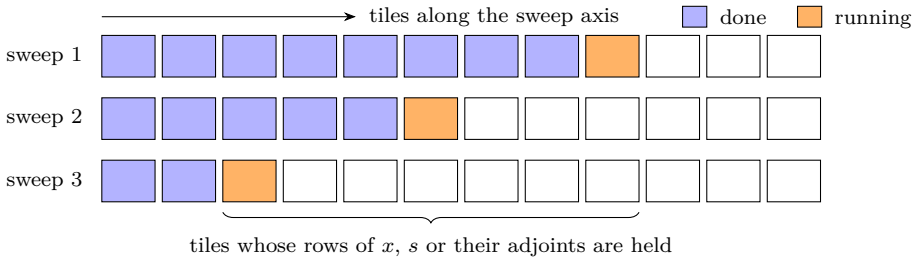


Figure 3: Wavefront schedule (schematic). Sweep 1 evaluates the first interaction and stores x and s; sweep 2 evaluates the second interaction and its reverse; sweep 3 repeats the first interaction and differentiates it. The sweeps follow each other along one axis, so state is held only for the tiles between the last and the first sweep. In the plain schedule each sweep finishes before the next begins, and the state of every atom is held.

Storing what the next interaction reads. Instead of the features h after the first product, FastMACE stores $x = Ah$ and $s = B_z h$, the two quantities the second interaction reads: 512 and 128 float32 values per atom. The three sweeps are then:

1. evaluate the first interaction without a tape and store x and s;
2. evaluate the second interaction and differentiate, giving direct edge derivatives and the adjoints g^x, g^s;
3. recompute the first interaction and differentiate the surrogate $\Phi_T = \sum_{i \in T} \big[\varepsilon_i^{(1)} + \langle g_i^x, x_i \rangle + \langle g_i^s, s_i \rangle\big]$ with the adjoints held fixed.

Wavefront and row reuse. A row of s can be overwritten by its adjoint as soon as its own tile has run, and a row of x once its last reader has run. A wavefront schedule interleaves the three sweeps along one axis of the tile grid, so that only a band of tiles holds state at any time (Figure 3). The state can live on the device or in host memory. Symmetrix-XL keeps its inter-layer state on the device and reaches larger systems by spreading them over GPUs; FastMACE has no multi-GPU path and reaches them by moving the state to the host.

Coordinates. In boxes larger than about 64 Å float32 positions lose enough precision to produce visible force errors on a perfect crystal (1.3×10^{-3} eV/Å at 1,200 Å). Edge vectors are therefore formed from float64 positions in a tile-local frame, which reduces that residual to 4×10^{-6} eV/Å.

10 Numerical validation

Acceptance rule. Production results are compared with a float64 reference. The float32 result of the reference implementation (MACE under ASE [19]) serves as the yardstick. For energy per atom, the largest force component and the largest stress component, each over the tested configurations,

$$e_{\text{production}} \;\le\; \max\!\big(3\, e_{\text{float32}},\; f\big), \tag{9}$$

with floors f of 0.01 meV/atom, 10^{-5} eV/Å and 10^{-7} eV/Å^3. The test configurations are $3 \times 3 \times 3$ supercells of $SrTiO_3$ (135 atoms), NaCl (216) and $NbCl_5$ (1,944), rattled by 0.05 Å and strained by $\pm 2\,\%$, hydrostatically and with the full cell, three configurations each.

		Energy (10^{-3} meV/atom)		Forces (10^{-5} eV/Å)		Stress (10^{-7} eV/Å^3)	
GPU and date	Structure	prod.	ref.	prod.	ref.	prod.	ref.
H200, 9 Nov	$SrTiO_3$	1.79	0.37	2.71	3.19	1.25	3.02
	NaCl	0.09	0.14	0.44	0.48	0.39	0.50
	$NbCl_5$	0.40	0.23	7.90	8.23	0.70	0.58
A100-SXM4, 11 Nov	$SrTiO_3$	2.13	0.22	2.76	2.95	1.40	2.57
	NaCl	0.14	0.13	0.41	0.47	0.41	0.43
	$NbCl_5$	0.64	0.11	8.09	8.07	0.84	0.67

Table 3: Largest errors against the float64 reference over the test configurations, for the production path ("prod.") and for the float32 reference implementation ("ref."). The A100 rows use the hybrid tensor-core product. The floors of Eq. (9) are 10, 1 and 1 in the units of the three column pairs.

History of the threshold. The factor was 2 until 7 November 2026. It was raised to 3 after one A100 configuration exceeded the limit in a single quantity ($NbCl_5$ stress, 1.8×10^{-7} against 1.2–1.5 $\times 10^{-7}$ eV/Å^3). The current A100 configuration passes that check at a ratio near 1.

Results. Table 3 gives the recorded errors. In forces the production path is at or below the float32 error of the reference (ratios 0.85 to 1.00). In stress it is between 0.41 and 1.25 times that error; the A100 with three TF32 products instead of the hybrid reaches 1.31. The energy is different. Its error is up to ten times that of the float32 reference (2.1×10^{-3} against 0.2×10^{-3} meV/atom on $SrTiO_3$), and it passes through the absolute floor of 0.01 meV/atom, not through the factor of three.

Against the published numbers of Symmetrix-XL. For the same model and crystal in float32, Symmetrix-XL reports differences from the reference implementation of 1.2×10^{-5} eV/atom in energy (0.012 meV/atom), 6.5×10^{-5} eV/Å in forces and 8.8×10^{-8} eV/Å^3 in stress [2]. These are not directly comparable with Table 3: they are differences from the reference on that paper's configurations, ours are errors against float64 on small rattled cells. The benchmark that first compared the two engines on one machine (3 November) measured force differences from MACE with cuEquivariance in float32 of 0.8×10^{-5} eV/Å for FastMACE and 1.7×10^{-5} eV/Å for Symmetrix-XL.

Energy conservation. In an NVE run of NaCl (216 atoms, 10 ps) the energy drift per atom was -4×10^{-6} meV/ps with the Chebyshev tables and 9×10^{-7} meV/ps without them. This record is from 2 November and predates the spline weights and the packed forward table. For comparison, Symmetrix-XL reports -9×10^{-7} meV/ps per atom over 100 ps on the 5,000-atom benchmark [2], a run we have not yet matched. For the tiled path the only record is a 1 ps run of 40,000 atoms at 600 K (0.002 meV/ps per atom), which is too short to separate drift from fluctuation.

What the check does not show. Passing establishes numerical agreement with the reference on those configurations. It is not a bound over all geometries, not a guarantee for long trajectories, and says nothing new about agreement with quantum-mechanical data. No relaxation, phonon, transport or stability benchmark has been run with FastMACE; Symmetrix-XL, by contrast, reproduces the published Matbench Discovery results [21] and three further workflows with an unchanged checkpoint. Reduced-precision storage of the tiled state (float16) fails the rule by factors of 12 to 51 and is offered only as a capacity fallback.

GPU	Date of record	$SrTiO_3$, 5,000 atoms		$NbCl_5$, 1,944 atoms
		ms per step	μs per atom and step	ms per step
H200	9 Nov 2026	3.40 / 3.37	0.68	1.09
A100-SXM4-80GB	11 Nov 2026	6.24 / 6.23	1.25	1.67
RTX 4000 Ada	10–11 Nov 2026	17.67 / 17.76	3.5	3.88 / 3.90
RTX 5060 Ti	10–11 Nov 2026	$\approx$ 15.4	3.1	$\approx$ 3.42

Table 4: Recorded steady-state MD step times (paired protocol). Each row comes from its own adoption study and source revision; this is not a controlled comparison of the GPUs.

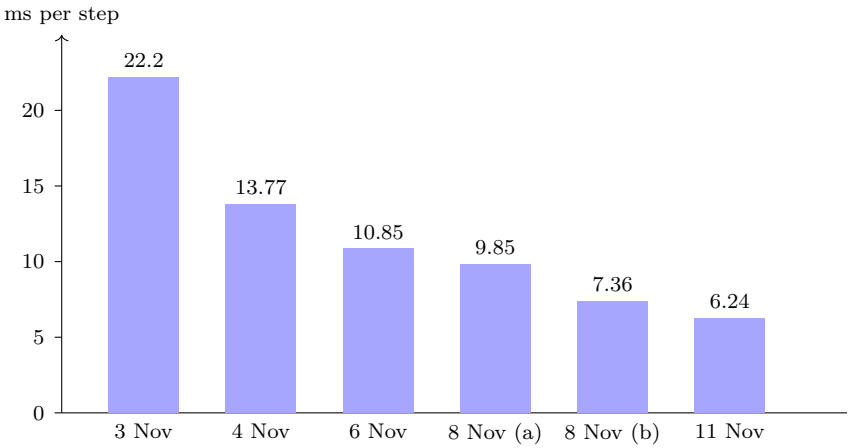


Figure 4: Recorded MD step time on the A100-SXM4-80GB for successive default configurations ($SrTiO_3$, 5,000 atoms). The values come from different jobs and are not paired measurements. Main changes between bars: custom second-interaction kernels and degree-5 tables (4 Nov); staging and fused first interaction (6 Nov); spline weights, receiver-major forward and composed first-interaction maps (8 Nov; bars a and b are the two sides of that paired study); packed table, generated contraction and hybrid product (11 Nov).

11 Performance

Protocols. Unless stated otherwise: OMAT-0 medium, float32, $SrTiO_3$ with 5,000 atoms, Langevin dynamics at 300 K, 1 fs time step, 0.5 Å skin. Two protocols appear. The *paired protocol* warms each engine up for 200 steps and times six blocks of 256 steps, synchronised at block boundaries, alternating between the two sides in one process; paired numbers are the two build orders. The *benchmark protocol*, used for comparisons with other engines on $SrTiO_3$, starts from rest, discards 5 steps and times 20, in three fresh processes. Integration and neighbour maintenance are included in both; construction, compilation and warm-up are not.

11.1 Step times

Table 4 gives the recorded times. Figure 4 shows how the A100's step time fell as defaults were adopted between 3 and 11 November 2026.

11.2 Effect of individual changes

Table 5 lists the paired measurements behind the main mechanisms. The local kernel saving and the whole-step saving agree where both were measured: the generated contraction saved 97 μs of kernel time on the H200 and 84–93 μs per step.

Change	GPU	Date	Step time ratio (two orders)
Radial tables (first evidence)	A100 PCIe	3 Nov	0.902 / 0.901
Reverse rebuilds weights from the table	A100	4 Nov	0.943 / 0.943
Factored reverse	A100	4 Nov	0.938 / 0.940
Fused first-interaction forward and reverse	A100	5 Nov	0.966 / 0.962
Spline weights, receiver-major forward, composed maps (ten switches)	A100	8 Nov	0.747 / 0.747
Edges sorted by length	H200	8 Nov	0.981 / 0.979
Packed forward table	A100	9 Nov	0.956 / 0.955
	RTX 4000 Ada	11 Nov	0.904 / 0.903
Generated symmetric contraction	H200	9 Nov	0.976 / 0.973
	A100 PCIe	9 Nov	0.969 / 0.965
Shared geometry sums in the forward	A100	10 Nov	0.976 / 0.981
Hybrid TF32 / FP16 product	A100	11 Nov	0.990 / 0.988

Table 5: Paired step-time ratios (new over old, below 1 is faster) from the studies on which each change was adopted. The baselines differ from row to row, so the ratios do not multiply to the totals of Figure 4.

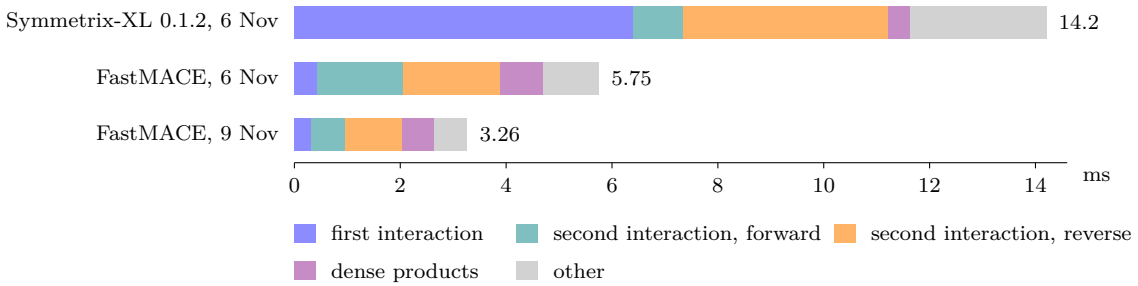


Figure 5: Kernel time of one force call on the H200 by stage ($SrTiO_3$, 5,000 atoms), from three separate profiles: Symmetrix-XL 0.1.2 under a tracing profiler on 6 November, FastMACE on the same day, and FastMACE on 9 November (Table 1). "Other" includes the symmetric contractions.

11.3 Comparison with Symmetrix-XL

Table 6 collects the runs in which both engines were measured on one machine in November. On 3 November FastMACE was 6 % slower on an A100; its radial tables were not yet enabled on that GPU. On the H200 two days later FastMACE was 2.6 times faster, and the ratio held for version 0.1.2. With the smaller model of the same family the ratio was 5.4, because Symmetrix-XL gained only a factor 1.27 from the smaller model where FastMACE gained 2.7. FastMACE used more GPU memory in every run: 1.6 to 1.8 times on the RTX 5060 Ti, and 4.5 times in the A100 run of 3 November, which predates the streamed kernels of Section 6. Automatic differentiation keeps intermediates that a hand-scheduled reverse pass does not.

Figure 5 shows where the difference arose. On 6 November Symmetrix-XL spent 6.4 ms in the first interaction, which FastMACE had already reduced to 0.4 ms by the rewrite of Section 4, and 3.9 ms in the reverse of the second interaction against 1.8 ms. Its forward of the second interaction was the faster one, 0.95 against 1.62 ms, and FastMACE adopted that layout two days later.

Since then. FastMACE's step has fallen from 5.72 to 3.40 ms on the H200 and from 24.4 to about 15.4 ms on the RTX 5060 Ti. We have not rerun Symmetrix-XL since 9 November, and its later releases may be faster, so these numbers do not give a current ratio.

GPU	Date	Version, model, system	ms per step			GPU memory
			S-XL	FastMACE	Ratio	S-XL / FastMACE
A100 PCIe	3 Nov	OMAT-0 medium, $SrTiO_3$ (5,000)	25.0	26.5	0.95	1.2 / 5.4 GB
H200	5 Nov	0.1.1, same	14.86	5.66	2.63	–
H200	6 Nov	0.1.2, same	14.57	5.72	2.54	–
H200	9 Nov	0.1.2, OMAT-0 small, $SrTiO_3$ (5,000)	11.48	2.12	5.4	–
RTX 5060 Ti	5 Nov	0.1.1, OMAT-0 medium, $SrTiO_3$ (5,000)	50.8	24.4	2.1	767 / 1,381 MiB
RTX 5060 Ti	5 Nov	0.1.1, OMAT-0 medium, NaCl (216)	3.58	1.15	3.1	455 / 720 MiB

Table 6: Same-machine Langevin runs of Symmetrix-XL ("S-XL", under LAMMPS) and FastMACE, each with the FastMACE defaults of its date. The ratio is the Symmetrix-XL time over the FastMACE time. A100 and H200 rows use the benchmark protocol; the RTX 5060 Ti rows time 300 and 3,000 steps after 100 and 500. The first row is from an external benchmark that does not state the version. In the H200 row of 5 November FastMACE ran with its A100 settings, which became its H200 defaults afterwards; with the generic settings it had until then the step took 13.1 ms (ratio 1.13). In the OMAT-0 small row the FastMACE time comes from a separate run of 8–9 November.

Engine	Model	ms per step, NaCl / $NbCl_5$		Date
		other engine	FastMACE	
ASE 3.27 with MACE's calculator (cuEquivariance)	MH-1 [22]	44.1 / 135.2	2.99 / 21.5	24 Oct
TorchSim 0.6.2 [23], best compile mode	OMAT-0 medium	16.3 / 39.4	1.62 / 9.79	25 Oct
ALCHEMI Toolkit 0.2.0 [24]	OMAT-0 medium	15.8 / 27.3	1.65 / 9.91	26 Oct

Table 7: Recorded Langevin step times of general engines and of FastMACE on an RTX 5060 Ti, for NaCl (216 atoms) and $NbCl_5$ (1,944 atoms), each pair on the same day and machine.

Its published numbers. For this model and crystal, the Symmetrix-XL paper reports 6.15 μs per atom and step on an A100-SXM4 80 GB at 1.25 million atoms, and 6.41 μs in tiled mode at 11.2 million atoms [2]. FastMACE's recorded values on the same GPU model are 1.25 μs per atom and step at 5,000 atoms and 2.1 μs per atom for a tiled force call at 0.98 million atoms (Section 11.5). The sizes, the step loop and the timing protocol differ, so this is an indication, not a measured ratio. A matched run at both sizes is the main missing experiment.

11.4 Comparison with general engines

Table 7 collects the recorded comparisons with engines that run MACE through its own modules. *All of them were made in October with earlier versions of FastMACE*, and the other engines have moved on as well. With that caveat, FastMACE was between 2.8 and 15 times faster per step on these small cells. We have no measurement against the LAMMPS interface with cuEquivariance [25], which is the baseline of the Symmetrix-XL paper.

Atoms	ms per step	μs per atom and step	Date of record
5,000	15.4	3.1	10–11 Nov
20,480	64.9	3.2	11 Nov
40,000	127.8	3.2	11 Nov
69,120	222–224	3.2	11 Nov
214,375	1,526	7.1	9–10 Nov

Table 8: MD step time of the whole-system path against system size on an RTX 5060 Ti (16 GB), $SrTiO_3$. The last row is the largest system that fits and runs with memory-saving rewrites.

Path	GPU	Atoms ($SrTiO_3$)	Time
Whole system, MD	RTX 5060 Ti, 16 GB	214,375	1.53 s per step
Whole system, MD	RTX 4000 Ada, 20 GB	296,595	2.04 s per step
Tiled, device state	A100-SXM4-80GB	975,560	2.04 s (2.1 μs/atom)
Tiled, device state	RTX 5060 Ti	975,560	6.1 s (6.2 μs/atom)
Tiled, wavefront, host state	RTX 4000 Ada, 20 GB, 251 GiB host	179,685,000	1,773 s (9.9 μs/atom)
Tiled, wavefront, host state	A100-SXM4-80GB, 231 GB host limit	320,000,000	3,585 s (11.2 μs/atom)
Tiled, wavefront, host state in float16 (fails the rule)	A100-SXM4-80GB	397,535,000	3,913 s (9.8 μs/atom)

Table 9: Largest recorded systems per path (9–11 November 2026). The tiled rows are single force calls, not MD steps; 975,560 atoms is the largest device-state size on record, not a limit.

11.5 Larger systems

Neighbour search. The pairwise screen of the rebuild grows with the square of the atom count. On an RTX 5060 Ti a rebuild takes 25 ms at 20,480 atoms and 2.1 s at 214,375 atoms; the cell-list screen brings these to 8 and 73 ms with a bit-identical list. In MD the step time ratio was 0.98 at 20,480 atoms and 0.96 at 40,000 atoms.

Cost against size. With the cell-list screen the whole-system path scales linearly until memory runs short (Table 8): on the RTX 5060 Ti the step costs 3.1 to 3.2 μs per atom from 5,000 to 69,120 atoms. Near the largest system that fits, the planner turns on rewrites that recompute instead of store, and the cost rises to 7.1 μs per atom. The RTX 4000 Ada behaves the same way, from 3.5 μs per atom at 5,000 atoms to 6.9 at 296,595.

Capacity. Table 9 gives the largest recorded systems. The whole-system path reaches 13,000 to 15,000 atoms per GB of device memory; the values Symmetrix-XL reports for its standard mode correspond to 15,000 to 16,000 [2]. The tiled path with the wavefront schedule and host state has evaluated 180 million atoms on a 20 GB card and 320 million on an 80 GB card in float32. In the first case the device was the limit and the host held 129 GB, or 0.7 kB per atom; in the second the host limit of 231 GB was the constraint. A 398-million-atom call stored the state in float16 and is therefore not a validated float32 result. These are single force calls. Symmetrix-XL reports 11.2 million atoms on one A100 with all state on the device, and 703 million atoms in MD on 64 GPUs [2].

Capturing the compiled tile bodies in CUDA graphs reduced the A100's tiled force call at 975,560 atoms from 3.59 to 2.04 s. The tiled path then costs 2.1 μs per atom against 1.25 for the whole-system path at 5,000 atoms.

Model	GPU	Structure (atoms)	ms per step	Date
OMAT-0 small	H200	$SrTiO_3$ (5,000)	2.12	9 Nov
OMAT-0 small	A100-SXM4	$SrTiO_3$ (5,000)	4.19	9 Nov
MACE-OFF23 S / M / L	A100-SXM4	water (648)	1.49 / 2.60 / 6.92	8 Nov
MACE-POLAR-1 S / M / L	A100-SXM4	NaCl (216)	2.29 / 4.23 / 7.71	8 Nov
MACE-POLAR-1 S / M / L	A100-SXM4	$NbCl_5$ (1,944)	7.77 / 16.38 / 29.06	8 Nov

Table 10: Recorded MD step times for other models with their own defaults: OMAT-0 small, the organic force fields MACE-OFF23 [26] in three sizes (S, M, L), and the polarisable MACE-POLAR-1 models.

11.6 Other models

The mechanisms are not specific to OMAT-0 medium, although the defaults differ by model. Table 10 lists recorded step times for other MACE families under their own defaults; each configuration passed the acceptance rule of Section 10. No reference implementation was timed on the same machines, so the table shows that the path runs, not how much it gains.

12 Related work

Kernels for equivariant tensor products. e3nn [5] provides the general language of MACE's operations, and cuEquivariance [11] optimised implementations of them. OpenEquivariance [15] and FlashTP [27] generate tensor-product kernels that fuse the Clebsch–Gordan contraction with the aggregation over neighbours. FastMACE builds on cuEquivariance and OpenEquivariance. Sobek [16] introduced the streaming schedule for equivariant convolutions that both our second-interaction kernels and Symmetrix-XL's descend from. These libraries accelerate one operation and leave the surrounding model program to the framework.

Inference engines for MACE. Symmetrix [28] evaluates MACE models natively, and Symmetrix-XL [2] extends it with streamed edges, per-model generated kernels and tiling. Table 11 compares its design with ours. Each approach pays for what it gains. A native evaluator stores no intermediates for automatic differentiation, which is why it uses less memory, and it inherits LAMMPS's integrators, domain decomposition and CPU and AMD back ends. Keeping MACE's modules means that every part FastMACE has not rewritten still runs through the reference code, that a rewrite can be switched off and compared in one process, and that new MACE variants run before they are optimised. TorchSim [23] and the ALCHEMI Toolkit [24] run MACE through its own modules with batched or compiled execution.

Deployment of other equivariant potentials. The NequIP and Allegro infrastructure compiles models and uses optimised tensor-product kernels [29, 30], and strictly local models have been scaled to very large systems [31]. chemtrain-deploy [32] and the parallel algorithm of Park et al. [33] run message-passing potentials with domain decomposition in LAMMPS [25, 34]. A different route to speed is a cheaper model, as in the neuroevolution potentials of GPUMD [35]; FastMACE and Symmetrix-XL instead keep the trained model and change its execution.

Tables and tensor cores. Tabulating embedding networks is used for Deep Potential models [12]. Splitting float32 operands to recover accuracy on tensor cores is due to Ootomo and Yokota [17]; the numerical behaviour of the hardware is analysed in [36]. Our hybrid product replaces two of the three passes by one half-precision pass.

Aspect	Symmetrix-XL	FastMACE
Model	Checkpoint converted to its own format; native evaluator	MACE's PyTorch modules, rewritten in a private copy
Kernels	Generated per model, precision and target; cached	General kernels chosen by a planner per GPU; one generated contraction
Forces	Hand-scheduled reverse stages	Automatic differentiation with custom reverse kernels
Radial functions	Cubic Hermite splines, 256 nodes; forces from the same spline	Chebyshev tables and banded cubic splines; optional packed forward table
Dense products	float32	Tensor cores (three TF32 products, or TF32 with FP16)
MD loop	LAMMPS or ASE	Own GPU-resident engine in captured graphs
Large systems	Tiled, three sweeps, state on the device; MPI over GPUs	Tiled, three sweeps, reduced state, wavefront, optional host state; one GPU
Hardware	NVIDIA GPUs and CPUs; a code path for AMD GPUs	NVIDIA GPUs
Validation	Differences from the reference; 100 ps NVE; published downstream benchmarks	Numeric rule per rewrite on three crystals; 10 ps NVE
Code	Public	Private at the time of writing

Table 11: Design of Symmetrix-XL and FastMACE. The Symmetrix-XL column is taken from its paper [2], its documentation as read on 4 November 2026, and our profile of version 0.1.2.

13 Origin of the mechanisms

Table 12 states where each mechanism came from, as far as the project's records show. Three categories are used: *vendored* code, an *adapted* idea with our own implementation, and *own* work. "Own" describes how a part arose in this project, not priority: where another engine has an equivalent, the table says so. A number of ideas reached the project through research reports written by large language models at the author's request; these are marked. For a few parts the records do not say where the idea came from.

14 Limitations and open work

- **No consolidated benchmark.** Step times, comparisons and accuracy numbers come from separate studies. A single run of one revision on all four GPUs, with the reference implementation, the LAMMPS interface with cuEquivariance and a current Symmetrix-XL at 5,000 and at about one million atoms, is still to be done.
- **No downstream validation.** The numerical check covers energies, forces and stresses on small rattled cells. Relaxations, phonons, transport and long trajectories have not been compared with the reference, as Symmetrix-XL has done.
- **Energy accuracy.** The energy error is several times that of the float32 reference and passes through an absolute floor of 0.01 meV/atom.
- **Energy and forces with the packed table.** They are not consistent to round-off, and the energy drift of that path has not been measured. The one drift record predates it and covers 216 atoms for 10 ps.

Mechanism	Origin
Receiver-streaming convolution kernels	Vendored: OpenEquivariance pull request 223 [14], after Sobek [16]
Three-product TF32 kernel	Vendored: CUTLASS [10]; method of [17]
Symmetric contraction, remaining tensor operations	Vendored: cuEquivariance [11]
Weights computed inside the convolution; direct spline weights; receiver-major forward	Adapted from Symmetrix-XL [2]
Three-sweep tiled evaluation	Adapted from Symmetrix-XL [2]
Aggregate before projecting (first interaction)	Adapted: the density trick [3]; proposed for this code in an LLM research report
Generated and factored symmetric contraction; shared geometry sums; packed table	Adapted: proposed in LLM research reports
Horner form of the reverse; receiver's adjoint row staged; cell-list screen	Adapted: proposed in LLM-written idea lists and a code review; the cell list is standard
Storing x and s; host state; per-tile neighbour lists	Adapted: outside idea lists; own form
Captured CUDA graphs; Verlet list	Adapted: standard techniques [18, 20]
Chebyshev radial tables and the acceptance test	Own. Tabulation is established [12]; Symmetrix-XL splines and gates the same networks [2]
Reverse without stored weights; factored reverse	Own, on the vendored kernel body. Symmetrix-XL's reverse also recomputes its radial factors [2]
Fused first-interaction forward and reverse; folded linear maps	Own
Hybrid TF32 / FP16 product	Own
Pipelined step, overlapped rebuild, noise batches, GPU pair search	Own
Wavefront schedule, row reuse, tile-local coordinates	Own
GPU-resident integrator; padded edge capacities; receiver-sorted edges	Not recorded

Table 12: Origin of the mechanisms, from the project's provenance notes. "Own" includes work done in LLM coding sessions directed by the author.

- **One GPU, NVIDIA only, one structure at a time.** There is no multi-GPU or CPU path. The largest systems are reached through host memory, as single force calls of half an hour to an hour, not as MD.
- **Memory.** FastMACE used 1.6 to 1.8 times the GPU memory of Symmetrix-XL in the runs of 5 November.
- **One model and one structure for timing.** Other MACE families run (Table 10) but were not compared with the reference here.
- **Sampled numerical checks.** The table test and the acceptance rule examine finite sets of distances and configurations.
- **No run-time range guard** for the hybrid tensor-core product.
- **Start-up cost** is excluded from all timings. Recorded setup times for the benchmark

were 33 to 48 s with warm caches and 94 to 110 s on a first run; in the one record that separates it, fitting the radial tables took about 35 s of a first build.

- **Remaining headroom.** On the A100 the two largest kernels are bound by one arithmetic unit, and paths around it were measured and closed: a half-precision path for distant edges failed the accuracy rule, and a double-precision path cost more registers than it saved time. Further gains at this model size are expected to be a few per cent each.

15 Conclusion

FastMACE shows how far the evaluation of a fixed equivariant potential can be reorganised without leaving the framework it was trained in. Its choices follow the structure of the calculation: use species-only inputs to aggregate before projecting, replace scalar radial networks by tested tables, differentiate streamed contractions directly, keep only what the next consumer reads, and run complete MD steps on the device. For the model studied here the recorded step time on an A100 fell from 22 to 6.2 ms between 3 and 11 November 2026, under an explicit accuracy rule, and in the same-machine runs on record FastMACE was 2.5 to 2.6 times faster than Symmetrix-XL on this benchmark from 5 November on. Symmetrix-XL remains ahead in validation, in memory use and in reach across hardware and system size. A consolidated benchmark against it and a downstream validation are the next steps.

Acknowledgements

Part of the measurements were run on the Alex cluster of NHR@FAU.

Use of AI tools. Large language models were used throughout this project. The implementation and the experiments were carried out in LLM coding sessions directed by the author; several mechanisms were first proposed in LLM-written research reports (Table 12); and this manuscript was drafted with LLM assistance from the project's records.

Code and data availability

The measurements reported here are recorded, with source revisions, in the project repository (Appendix A).

References

[1] Ilyes Batatia, Dávid Péter Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2206.07697.

[2] Lei Fu, Zihui Feng, Yongheng Li, Hongwei Du, Xin He, Junyi Wu, Kejie Bao, Yueyu Zhang, Zeyu Deng, Ziheng Lu, and Bonan Zhu. Train for accuracy, execute at scale: Architecture-preserving inference for equivariant atomistic foundation models. *arXiv preprint arXiv:2610.01036*, 2026.

[3] Ralf Drautz. Atomic cluster expansion for accurate and transferable interatomic potentials. *Physical Review B*, 99(1):014104, 2019. doi: 10.1103/PhysRevB.99.014104.

[4] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P. Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E. Smidt, and Boris Kozinsky. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1):2453, 2022. doi: 10.1038/s41467-022-29939-5. arXiv:2101.03164.

[5] Mario Geiger and Tess Smidt. e3nn: Euclidean neural networks. *arXiv preprint arXiv:2207.09453*, 2022.

[6] Ilyes Batatia, Philipp Benner, Yuan Chiang, Alin M. Elena, Dávid P. Kovács, Janosh Riebesell, Xavier R. Advincula, Mark Asta, Matthew Avaylon, William J. Baldwin, et al. A foundation model for atomistic materials chemistry. *The Journal of Chemical Physics*, 163:184110, 2025. doi: 10.1063/5.0297006. arXiv:2401.00096.

[7] Luis Barroso-Luque, Muhammed Shuaibi, Xiang Fu, Brandon M. Wood, Misko Dzamba, Meng Gao, Ammar Rizvi, C. Lawrence Zitnick, and Zachary W. Ulissi. Open materials 2024 (OMat24) inorganic materials dataset and models. *arXiv preprint arXiv:2410.12771*, 2024.

[8] Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024. doi: 10.1145/3620665.3640366.

[9] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019. doi: 10.1145/3315508.3329973.

[10] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Aditya Atluri, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS. Software, https://github.com/NVIDIA/cutlass, 2023. Version 4.8.1 used here.

[11] NVIDIA. cuEquivariance. Software, https://github.com/NVIDIA/cuEquivariance, 2026. Version 0.12.0 used here.

[12] Denghui Lu, Wanrun Jiang, Yixiao Chen, Linfeng Zhang, Weile Jia, Han Wang, and Mohan Chen. DP compress: A model compression scheme for generating efficient deep potential models. *Journal of Chemical Theory and Computation*, 18(9):5559–5567, 2022. doi: 10.1021/acs.jctc.2c00102. arXiv:2107.02103.

[13] Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.

[14] Paul Fuchs. Add receiver-streaming UVU convolution kernel. OpenEquivariance pull request 223, https://github.com/PASSIONLab/OpenEquivariance/pull/223, 2026.

[15] Vivek Bharadwaj, Austin Glover, Aydin Buluc, and James Demmel. An efficient sparse kernel generator for O(3)-equivariant deep networks. In *Proceedings of the SIAM Conference on Applied and Computational Discrete Algorithms (ACDA25)*, 2025. arXiv:2501.13986.

[16] Vladimir Chorošajev and Cédric Bény. Sobek: Streaming equivariant tensor product convolutions. *arXiv preprint arXiv:2607.18074*, 2026.

[17] Hiroyuki Ootomo and Rio Yokota. Recovering single precision accuracy from tensor cores while surpassing the FP32 theoretical peak performance. *The International Journal of High Performance Computing Applications*, 36(4):475–491, 2022. doi: 10.1177/10943420221090256. arXiv:2203.03341.

[18] NVIDIA. CUDA C++ programming guide: CUDA graphs. Documentation, https://docs.nvidia.com/cuda/cuda-programming-guide/, 2026.

[19] Ask Hjorth Larsen, Jens Jørgen Mortensen, Jakob Blomqvist, Ivano E. Castelli, Rune Christensen, Marcin Dułak, Jesper Friis, Michael N. Groves, Bjørk Hammer, Cory Hargus, et al. The atomic simulation environment—a Python library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27):273002, 2017. doi: 10.1088/1361-648X/aa680e.

[20] Loup Verlet. Computer "experiments" on classical fluids. I. thermodynamical properties of Lennard-Jones molecules. *Physical Review*, 159(1):98–103, 1967. doi: 10.1103/PhysRev.159.98.

[21] Janosh Riebesell, Rhys E. A. Goodall, Philipp Benner, Yuan Chiang, Bowen Deng, Gerbrand Ceder, Mark Asta, Alpha A. Lee, Anubhav Jain, and Kristin A. Persson. Matbench discovery – a framework to evaluate machine learning crystal stability predictions. *arXiv preprint arXiv:2308.14920*, 2023.

[22] Ilyes Batatia, Chen Lin, Joseph Hart, Elliott Kasoar, Alin M. Elena, Sam Walton Norwood, Thomas Wolf, and Gábor Csányi. Cross learning between electronic structure theories for unifying molecular, surface, and inorganic crystal foundation force fields. *arXiv preprint arXiv:2510.25380*, 2025.

[23] Orion Cohen, Janosh Riebesell, Rhys Goodall, Adeesh Kolluru, Stefano Falletta, Joseph Krause, Jorge Colindres, Gerbrand Ceder, and Abhijeet S. Gangan. TorchSim: An efficient atomistic simulation engine in PyTorch. *AI for Science*, 1(2):025003, 2025. doi: 10.1088/3050-287X/ae1799. arXiv:2508.06628. Software version 0.6.2 used here.

[24] NVIDIA. ALCHEMI toolkit. Software, version 0.2.0 used here. https://github.com/NVIDIA/nvalchemi-toolkit, 2026.

[25] Anders Johansson, Evan Weinberg, Christian R. Trott, Megan J. McCarthy, and Stan G. Moore. LAMMPS-KOKKOS: Performance portable molecular dynamics across exascale architectures. *arXiv preprint arXiv:2508.13523*, 2025.

[26] Dávid Péter Kovács, J. Harry Moore, Nicholas J. Browning, Ilyes Batatia, Joshua T. Horton, Yixuan Pu, Venkat Kapil, William C. Witt, Ioan-Bogdan Magdău, Daniel J. Cole, and Gábor Csányi. MACE-OFF: Short-range transferable machine learning force fields for organic molecules. *Journal of the American Chemical Society*, 147(21):17598–17611, 2025. doi: 10.1021/jacs.4c07099. arXiv:2312.15211.

[27] Seung Yul Lee, Hojoon Kim, Yutack Park, Dawoon Jeong, Seungwu Han, Yeonhong Park, and Jae W. Lee. FlashTP: Fused, sparsity-aware tensor product for machine learning interatomic potentials. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 33143–33156, 2025.

[28] William C. Witt. Symmetrix. Software, https://github.com/wcwitt/symmetrix, 2026.

[29] Chuin Wei Tan, Marc L. Descoteaux, Mit Kotak, Gabriel de Miranda Nascimento, Seán R. Kavanagh, Laura Zichi, Menghang Wang, Aadit Saluja, Yizhong R. Hu, Tess Smidt, Anders

Johansson, William C. Witt, Boris Kozinsky, and Albert Musaelian. High-performance training and inference for deep equivariant interatomic potentials. *Digital Discovery*, 2026. doi: 10.1039/D5DD00423C. arXiv:2504.16068.

[30] Seán R. Kavanagh, Chuin Wei Tan, Menghang Wang, Marc L. Descoteaux, Gabriel de Miranda Nascimento, Ulrik Unneberg, Laura Zichi, Francesco Libbi, Norma Rivano, Austin Glover, Vivek Bharadwaj, Anders Johansson, William C. Witt, Albert Musaelian, and Boris Kozinsky. Fast and accurate equivariant foundation models for atomistic simulation. *arXiv preprint arXiv:2607.28461*, 2026.

[31] Albert Musaelian, Anders Johansson, Simon Batzner, and Boris Kozinsky. Scaling the leading accuracy of deep equivariant models to biomolecular simulations of realistic size. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23)*, 2023. doi: 10.1145/3581784.3627041. arXiv:2304.10061.

[32] Paul Fuchs, Weilong Chen, Stephan Thaler, and Julija Zavadlav. chemtrain-deploy: A parallel and scalable framework for machine learning potentials in million-atom MD simulations. *arXiv preprint arXiv:2506.04055*, 2025.

[33] Yutack Park, Jaesun Kim, Seungwoo Hwang, and Seungwu Han. Scalable parallel algorithm for graph neural network interatomic potentials in molecular dynamics simulations. *Journal of Chemical Theory and Computation*, 20(11):4857–4868, 2024. doi: 10.1021/acs.jctc.4c00190. arXiv:2402.03789.

[34] Aidan P. Thompson et al. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022. doi: 10.1016/j.cpc.2021.108171.

[35] Zheyong Fan, Yanzhou Wang, Penghua Ying, Keke Song, Junjie Wang, Yong Wang, Zezhu Zeng, Ke Xu, Eric Lindgren, J. Magnus Rahm, et al. GPUMD: A package for constructing accurate machine-learned potentials and performing highly efficient atomistic simulations. *The Journal of Chemical Physics*, 157(11):114801, 2022. doi: 10.1063/5.0106617. arXiv:2205.10046.

[36] Massimiliano Fasi, Nicholas J. Higham, Mantas Mikaitis, and Srikara Pranesh. Numerical behavior of NVIDIA tensor cores. *PeerJ Computer Science*, 7:e330, 2021. doi: 10.7717/peerj-cs.330.

A Records and software

Software. The recorded runs used PyTorch 2.14.1 with CUDA 13.0, Triton 3.8.0, cuEquivariance 0.12.0, OpenEquivariance 0.7.0, mace-torch 0.3.16, e3nn 0.4.4, ASE 3.27.0, CUTLASS 4.8.1 and Python 3.10. Symmetrix-XL was run from its 0.1.1 and 0.1.2 wheels with a LAMMPS plugin built from the matching source, in float32 with its speed profile and a prepared device artifact.

Records. Table 13 names the record behind each table and figure. A record is a directory of the project repository that holds the job scripts, logs and results of one study.

Item	Record	Source revision
Profile (Table 1)	`step-limiters-20261109`	–
Accuracy, H200 (Table 3)	`sc0-op-20261109`	`5c5ade67`
Accuracy, A100 (Table 3)	`hybrid-gemm-20261111`	`4195be0d`
Step times (Table 4)	`sc0-op-20261109`, `hybrid-gemm-20261111`, `ada-5060-screen-20261110`, `step-limiters-20261109`	`5c5ade67`, `4195be0d`, `edfc0115`
Symmetrix-XL runs (Table 6)	`loup-a100` (validation note), `symmetrix-xl-h200-loup-20261105`, `sx012-profile-20261106`, `symmetrix-xl-h200-omat0-small-20261109`, `vram-compare-rtx5060ti-20261105`, `nacl-compare-rtx5060ti-20261105`	`7430fb51`, `d599cc70` (FastMACE sides of 3 and 6 Nov)
Stages (Figure 5)	`sx012-profile-20261106`, `step-limiters-20261109`	`d599cc70`
General engines (Table 7)	phase-4 validation note; TorchSim and ALCHEMI comparison notes of 25 and 26 October	`4c574d2`
Size (Table 8)	`ada-5060-screen-20261110`, `binned-screen-5060ti-20261111`, `shuffled-order-5060ti-20261111`, `rung-b-20261109`	`b951e804`, `f31d4f50`, `cd56800c`
Capacity (Table 9)	`rung-b-20261109`, `tiled-speed-20261111`, `tiled-capacity-20261109`	`cd56800c`, `d036568b`, `7e4fbf4c`
Other models (Table 10)	`off-polar-a100-20261108`, `h200-omat0-small-20261108`	`b4ec7053`
Radial tables (Section 5)	`packed-table-lab-20261109`, `first-interaction-20261111`	–

Table 13: Records behind the tables and figures. Revisions are abbreviated commit identifiers of the FastMACE repository; a dash means the record names several.